

Projet Cybersecurite-AP2 — Introduction à la sécurité informatique et aux attaques de mots de passe

Projet du 20/11/2025



**l'école d'ingénierie
informatique**



**l'école [tech]
de l'expertise digitale**

Participants :

Kaëlian BAUDELET
Florian GUILBERT
Thibault ROYER

**Atelier Cybersécurité — Deuxième
Année, 2025**

**EPSI — Ecole de l'ingénierie
informatique, Arras**

Projet encadré par :

Julien LE GALES

CONSIGNE DU PROJET

CyberSécurité – Gestion des accès et des habilitations

Ce projet s'inscrit dans le cadre de la cybersécurité ainsi que dans celui de l'épreuve E5 du BTS SIO.

Il doit être réalisé en groupe de deux ou trois étudiants maximum.

Chaque groupe passera à l'oral, présentera une démonstration de son application et un diaporama expliquant la démarche et les choix techniques retenus.

Votre mission est de créer une application Symfony sécurisée permettant :

- Aux utilisateurs de s'inscrire et de se connecter.
Les informations demandées sont : nom, prénom, adresse, ville et code postal.
Les saisies doivent être validées avant l'insertion dans la base de données à l'aide des Validation Constraints Reference de Symfony.
- Le mot de passe doit respecter l'ensemble des critères de sécurité préconisés (longueur minimale, complexité, non-compromission (NotCompromisedPassword), etc.).
- L'utilisateur doit pouvoir modifier son mot de passe, sans pouvoir réutiliser un ancien mot de passe.
- Le système de connexion doit être protégé contre les attaques par force brute.
- L'administrateur doit pouvoir gérer les comptes, rôles et habilitations : il pourra ajouter, modifier, supprimer ou suspendre des utilisateurs.
La liste des utilisateurs devra également permettre d'afficher l'historique complet des connexions (dates et heures).
- L'application doit être hébergée sur GitHub et conçue de manière à pouvoir être réutilisée comme base pour de nouveaux projets.

MODALITÉS DU PROJET

- Durée de rendu: 20/11/25
- Ressources :
 - Ressources documentaires numérique en ligne
 - Cours sur la Plateforme 360Learning
 - Bibliothèque ENI
- Objectifs et livrables :
 - Concevoir un site template respectant les normes données
 - Être hébergée sur GitHub.
 - Pouvoir gérer les comptes, rôles et habilitations en tant qu'administrateur.
 - Être protégé contre les attaques par force brute.
 - Pouvoir modifier son mot de passe SANS utiliser un ancien mot de passe.
 - Les mot de passe doivent respecter l'ensemble des critères de sécurité préconisés
 - S'inscrire ou se connecter
 - Un diaporama de présentation du projet
 - Ce dossier de présentation
- Outils utilisés
 - Php/Symfony
 - Trello/Jira
 - PhpMyAdmin
 - Discord
- Méthode de travail
 - Scrum
 - Méthode Agile

Table des matières

1. Introduction	08
1.1. Contexte du projet	08
1.2. Objectifs généraux	08
2. Présentation du projet.....	08
2.1. Description du site et du template.....	08
2.2. Fonctionnalités principales	08
2.3. Technologies utilisées.....	08
3. Méthode de travail.....	09
3.1. Approche agile.....	09
3.2. Cadre SCRUM	09
3.3. Organisation du travail et gestion du projet.....	10
4. Architecture et conception.....	10
4.1. Schéma WireFrame.....	10
4.2. Modèle de données (MCD).....	10
4.3. Gestions des rôles et habilitations.....	10
5. Développement et implémentation.....	11
5.1. Inscription & Connexion.....	11
5.2. Système d'administration.....	11
5.3. Déploiement sur Github.....	11
6. Sécurité du système.....	12
6.1. Protection de la connexion contre les attaques force brute.....	12
6.2. Politique de mot de passes et exigences de sécurité.....	12
6.3. Gestion de la réinitialisation de mot de passe.....	13
7. Test et validation.....	15
7.1. Tests Fonctionnels.....	15
7.2. Tests de sécurité	17
Conclusion.....	20
Ressources documentaires et références utilisés.....	21
Annexes.....	22

Projet AP2-Cybersecurite — Introduction à la sécurité informatique et aux attaques de mots de passe



Projet AP2-Cybersecurite — SN2 2025-2026
Atelier Cybersécurité

Table des Annexes

Le dossier de projet suivant contient plusieurs annexes regroupant plusieurs images, illustrations et schémas.

Annexe 1.....	23
Annexe 2.....	23
Annexe 3.....	24
Annexe 4.....	25
Annexe 5.....	25
Annexe 6.....	26
Annexe 7.....	26
Annexe 8.....	28
Annexe 9.....	28
Annexe 10.....	29
Annexe 11.....	29

Avant propos

L'ensemble du code, fichiers et éléments nécessaires à la reproduction de ce projet est disponible sur GitHub :
<https://github.com/kaelianbaudelet/projet-cybersecurite-ap2>.

Ce projet a été conçu dans un objectif pédagogique et technique, afin d'illustrer la mise en place d'un système complet incluant l'authentification et des notions de sécurité.

1. Introduction

1.1. Contexte du projet

Le projet consiste à faire une page web avec ce que l'on a vu dans les cours symfony. Cette interface contiendra un espace d'inscription et de connexion sécurisé (mot de passe respectant les critères de sécurité préconisés, haché). Ce même mot de passe doit pouvoir être modifié par les utilisateurs, par contre ils ne peuvent pas réutiliser un mot de passe qu'ils avaient choisi au préalable. Ensuite nous devons avoir un système pour contrer les forces brutes et enfin L'administrateur doit pouvoir gérer les comptes, les rôles et les habilitations. Ce projet s'inscrit dans le cadre de la cybersécurité ainsi que dans celui de l'épreuve E5 du BTS SIO.

1.2. Objectifs généraux

Notre objectif est de créer la base d'un site web que l'on pourra utiliser pour de futurs projets et nous permet de nous familiariser avec les pratiques de cybersécurité.

2. Présentation du projet

2.1. Description du site et du template

Le projet consistant à créer un site simple, nous avons mit le moins d'artifice possible en gardant le plus d'options flexible. C'est à dire que nous n'avons mit que les pages nécessaire pour répondre : La main page, les pages de logins / connexion, la page de profile et la page d'administration des utilisateurs & logs avec leur page de modifications. Le reste est libre à la modification.

2.2. Fonctionnalités principales

Pour la base de site, nous avons considéré de mettre tout ce qu'il y a de base pour chaque site, c'est à dire un système d'administrations et une création de compte. En mettant, en plus, une décoration bootstrap modifiable pour rendre le site présentable. La sécurité étant principale tâche du projet, nous avons mit en place, avec symfony, les bases pour la connexions. Le mot de passe devant respecter les normes données par l'ANSSI et la non compromition. Et les administrateurs ont accès à la modifications des utilisateurs sans pouvoir modifier les détails précis tel que la date de création du compte.

2.3. Technologies utilisées

Pour répondre aux demandes du projet, nous avons utilisé le framework symfony en version 6.4 (dernière version maintenue dans le temps), fiable et simple. Ainsi que phpMySQL pour la gérer la base de donnée.

De plus, nous avons utilisé les modules Symfony ci dessous:

- Security pour gérer l'auth, les rôles et les mots de passes.
- rate-limiter pour protéger la connexion des attaques par force brute.
- mailer pour envoyer des mails de connexions aux utilisateurs
- validator pour empêcher les mots de passes compromis et validées les données de formulaires

Pour l'organisation du projet nous avons aussi utilisé github en nous séparant le travail agilement (voir [annexe 1](#)) et Jira (voir [annexe 2](#)) pour formaliser et centraliser les tâches et sprint de la méthode SCRUM.

3. Méthode de travail

3.1. Approche agile

Pour mener à bien le développement du projet nous avons utilisés l'approche agile.

Reposant sur l'adaptabilité, l'amélioration continue et la communication.

Tout d'abord, par l'intermédiaire de gitHub nous avons pu développer la livraison incrémentale. Nous avons chacun créé nos fonctionnalités tout en mergeant sur la branche main une fois cette dernière finalisée. (voir [annexe 1](#))

Ensuite, pour gérer la communication nous avons utilisé Jira (voir [annexe 2](#)) pour nous tenir au courant de l'avancement de chacun ainsi que le canal de discussion Discord pour communiquer rapidement et facilement entre nous en cas de problème technique.

Des minis réunions ont aussi été organisés pour parler du projet, de son avancement & de ce que l'on pourrait faire pour améliorer ce dernier. Ces dernières passant notamment par le canal de discussion sus-mentionné.

Le canal de discussion et les réunions nous ont aussi permit de pouvoir revoir les priorités du projet afin de travailler sur le plus gros du travail au plus vite.

Pourquoi l'Agile ?

- Le développement comporte plusieurs fonctionnalités complexes (sécurité, rôles, reset password, brute force) : l'Agile permet d'en prioriser certaines et d'ajuster le plan en cours de route.
- Elle favorise une meilleure réactivité face aux contraintes techniques.
- Elle permet d'obtenir rapidement des versions testables (MVP) pour valider les features et en discuter pour les améliorer.

3.2. Cadre SCRUM

Pour continuer sur l'organisation nous avons utilisé le cadre SCRUM pour le projet.

Nous nous sommes répartis les rôles comme ceci:

Kaelian Baudalet : Developpeur / Product Owner

Thibault Royer : Scrum master / Developpeur

Florian Guilbert: Developpeur

Le Product Backlog regroupait l'ensemble des tâches du projet, par exemple :

- Système d'inscription
- Système de connexion
- Gestion des rôles et habilitations
- Protection brute force
- Reset password sécurisé
- Hébergement GitHub
- Sécurité

Chaque élément a été ajouté dans Trello/Jira et priorisé de notre côté.

Le projet a été séparé en 3 sprints:

Sprint 1 : Définition des priorités et attendus du projet

Sprint 2: Conception du projet

Sprint 3: Finissions, écriture du dossier & conception des diapositives.

Seul le sprint 2 a été répertorié sur le Jira, composant une majeure partie des tâches à faire et à prioriser. (voir [annexe 2](#) & [annexe 3](#))

3.3. Organisation du travail et gestion du projet

Pour ce projet nous nous sommes chacun donné des features:

Reset Password Sécurisé et Hébergement Github -> Florian Guilbert

Système de connexion, inscription et sécurité des mots de passes -> Kaelian Baudalet

Gestion des rôles et habilitation et Protection contre les attaques brute force -> Thibault Royer

Pour chaque feature, nous avons créé notre branche github (feature/"nom_de_la_feature") et nous ne mergions qu'une fois la feature finie. (voir [annexe 1](#))

4. Architecture et conception

4.1. Schéma WireFrame

Ne devait faire qu'un site de base simple, nous avons développer quatres wireframe:

- Page principale "accueil"
- Page de connexion
- Page de profil
- Page d'administration des utilisateurs & logs

Les [annexe 4](#), [annexe 5](#), [annexe 6](#) et [annexe 7](#) sont ces derniers respectivement.

4.2. Modèle de données (MCD)

Pour faire le modèle de donnée nous avons tout d'abord définis toutes les données dont nous allions avoir besoin. ([annexe 8](#))

Ensuite, nous sommes directement passé à la conception du MCD (voir [annexe 9](#))

4.3. Gestions des rôles et habilitations

Pour gérer les rôles et les habilitations nous avons déterminés 3 rôles:

- User
- Modérateur
- Administrateur

L'utilisateur étant une personne avec le minimum de droit, pouvant se connecter, le modérateur n'ayant, pour l'instant aucun droit et l'administrateur ayant le droit de voir la liste des utilisateur et de les gérer (modifier leurs informations, les suspendre, ...)

5. Développement et implémentation

5.1. Inscription & Connexion

Pour la mise en place du système, nous avons commencé par implémenter l'authentification afin de permettre aux utilisateurs de s'inscrire et de se connecter de manière sécurisée. Cette étape constitue la base sur laquelle toutes les autres fonctionnalités de gestion des accès et des habilitations seront construites.

Pour l'inscription, nous avons créé un formulaire avec Symfony Forms permettant de saisir plusieurs informations personnelles : nom, prénom, adresse, ville, code postal, ainsi qu'un mot de passe. Afin de garantir la fiabilité et la sécurité des données saisies, toutes les informations sont validées grâce aux **Validation Constraints** de Symfony, ce qui permet de prévenir les saisies incorrectes ou malveillantes.

C'est notamment ce que nous avons fait pour le formulaire d'inscription :

```
public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('firstName', TextType::class, [
            'label' => 'Prenom',
            'attr' => [
                'autocomplete' => 'given-name',
                'placeholder' => 'Marie',
            ],
        ])
        ->add('lastName', TextType::class, [
            'label' => 'Nom',
            'attr' => [
                'autocomplete' => 'family-name',
                'placeholder' => 'Curie',
            ],
        ])
        ->add('email', EmailType::class, [
            'label' => 'Email',
            'attr' => [
                'autocomplete' => 'email',
                'placeholder' => 'exemple@domaine.fr',
            ],
        ])
        ->add('address', TextType::class, [
            'label' => 'Adresse',
            'attr' => [
                'autocomplete' => 'street-address',
                'placeholder' => '1 rue de la Republique',
            ],
        ])
        ->add('city', TextType::class, [
            'label' => 'Ville',
            'attr' => [
                'autocomplete' => 'address-level2',
                'placeholder' => 'Paris',
            ],
        ])
        ->add('postalCode', TextType::class, [
            'label' => 'Code postal',
            'attr' => [
                'autocomplete' => 'postal-code',
                'placeholder' => '75001',
            ],
        ])
        ->add('agreeTerms', CheckboxType::class, [
            'mapped' => false,
            'constraints' => [
                new IsTrue([
                    'message' => 'Vous devez accepter les conditions d\'utilisation.',
                ]),
            ],
        ])
        ->add('plainPassword', PasswordType::class, [
            'mapped' => false,
            'attr' => ['autocomplete' => 'new-password'],
            'constraints' => [
                new NotBlank([
                    'message' => 'Veuillez saisir un mot de passe.',
                ]),
                new Length([
                    'min' => 12,
                    'minMessage' => 'Votre mot de passe doit contenir au moins {{ limit }} caracteres.',
                    'max' => 4096,
                ]),
                new Regex([
                    'pattern' => '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[\\W_]).+$/ ',
                    'message' => 'Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.',
                ]),
                new NotCompromisedPassword([
                    'message' => 'Ce mot de passe a ete compromis dans une fuite de donnees. Merci d\'en choisir un autre.',
                ]),
            ],
        ])
    }
```

Le mot de passe est soumis à des exigences strictes de sécurité, incluant une longueur minimale et la présence d'au moins une majuscule, une minuscule, un chiffre et un caractère spécial. De plus, nous avons intégré la vérification **NotCompromisedPassword**, qui utilise la base de données du site haveibeenpwned.com pour s'assurer que le mot de passe choisi n'a pas été compromis lors de fuites de données. Ces vérifications garantissent que les comptes créés sont protégés contre les attaques liées à des mots de passe faibles ou compromis.

```
#[Route('/register', name: 'app_register')]
public function register(Request $request, UserPasswordHasherInterface $userPasswordHasher, Security $security, EntityManagerInterface $entityManager): Response
{
    if ($this->getUser()) {
        return $this->redirectToRoute('app_accueil');
    }

    $user = new User();
    $form = $this->createForm(RegistrationFormType::class, $user);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $plainPassword = $form->get('plainPassword')->getData();
        $hashedPassword = $userPasswordHasher->hashPassword($user, $plainPassword);

        $entityManager->persist($user);
        $entityManager->flush();

        $user->setPassword($hashedPassword);
        $entityManager->flush();

        $this->emailVerifier->sendEmailConfirmation(
            'app_verify_email',
            $user,
            (new TemplatedEmail())
                ->from(new Address($this->mailerFromEmail, $this->mailerFromName))
                ->to((string) $user->getEmail())
                ->subject('Confirmez votre adresse email')
                ->htmlTemplate('registration/confirmation_email.html.twig')
        );

        $this->addFlash('success', 'Un email de verification vous a ete envoye. Pensez a confirmer votre adresse.');
```

```
        return $security->login($user, LoginFormAuthenticator::class, 'main');
    }

    return $this->render('registration/register.html.twig', [
        'registrationForm' => $form,
    ]);
}
```

Pour la connexion, nous avons utilisé le composant **Security** de Symfony. Les utilisateurs peuvent se connecter via leur email et leur mot de passe, et sont redirigés automatiquement vers leur page profil après une authentification réussie. Cette redirection permet de centraliser l'accès aux fonctionnalités personnelles et de sécuriser les interactions avec l'application. La combinaison d'un formulaire d'inscription sécurisé et d'un processus de connexion robuste assure que le système d'authentification repose sur des bases solides et protège efficacement les comptes utilisateurs contre les risques courants.

```
#[Route(path: '/login', name: 'app_login')]
public function login(AuthenticationUtils $authenticationUtils): Response
{
    if ($this->getUser()) {
        return $this->redirectToRoute('app_accueil');
    }

    $error = $authenticationUtils->getLastAuthenticationError();
    $lastUsername = $authenticationUtils->getLastUsername();

    return $this->render('security/login.html.twig', ['last_username' => $lastUsername, 'error' => $error]);
}
```

5.2. Système d'administration

Pour gérer le système de l'administration nous avons pensé à faire une page où tout les utilisateurs seraient répertoriés, permettant de modifier les informations rentrées par l'utilisateur (nom, prénom, adresse...) et surtout permettre de bloquer l'accès à la connexion de l'utilisateur en cochant la case "utilisateur suspect". De plus, l'interface permet de gérer les rôles des utilisateurs. (Images en [annexe 10](#))

Pour se faire, nous avons créé un form spécial puisque nous affichons un form, sur la même page, pour chaque utilisateur.

```
#[Route('/adm-users-list_editor', name: 'app_users_list_editor')]
public function usersListEditor(Request $request, EntityManagerInterface $em, FormFactoryInterface $formFactory): Response
{
    $users = $em->getRepository(User::class)->findAll();
    $forms = [];

    foreach ($users as $user) {
        $form = $formFactory->createNamed(
            'user_'.$user->getId(),
            AdminUserEditorType::class,
            $user
        );
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid()) {
            $em->flush();
            $this->addFlash('notice', "Utilisateur {$user->getEmail()} modifié");
            return $this->redirectToRoute('app_users_list_editor');
        }
        $forms[$user->getId()] = $form->createView();
    }

    return $this->render('admin/users_list_editor.html.twig', ['users' => $users, 'forms' => $forms,]);
}
```

Nous pouvons notamment voir que nous ne faisons pas directement de `createForm` mais que nous définissons tout d'abord une variable contenant tout les formulaires. Et que, en suite, pour chaque utilisateur nous créons un form déterminé par le nom `user_ "son_id"`. Afin de récupérer toutes les données présentes dans chaque formulaire et ne pas attribuer à un mauvais user les données entrées.

Nous avons décidé de faire de cette manière afin de permettre aux administrateurs d'administrer rapidement le site sans à avoir à charger une nouvelle page pour chaque utilisateur qu'il souhaite changer.

5.3. Déploiement sur Github

Nous avons travaillé sur GitHub en adoptant une méthode agile, en organisant notre développement par itérations et sprints. Pour cela, nous avons utilisé une stratégie de branches comprenant `main`, `dev` et des branches spécifiques pour chaque fonctionnalité (feature). Chaque membre de l'équipe avait la responsabilité d'implémenter certaines fonctionnalités sur sa branche dédiée lors des sprints.

The screenshot shows the GitHub web interface for the repository 'projet-cybersecurite-ap2'. The 'Branches' tab is selected, showing a list of branches organized into three sections: Default, Your branches, and Active branches. Each section contains a table with columns for Branch, Updated, Check status, Behind/Ahead, and Pull request.

Default				
Branch	Updated	Check status	Behind / Ahead	Pull request
main	16 hours ago	3 / 3	Default	

Your branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
dev	yesterday	2 / 3	2 0	

Active branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
features/loginLogs	16 hours ago	4 / 5	1 0	#5
dev	yesterday	2 / 3	2 0	
features/ajoutUpdatePassword	yesterday		8 1	
features/ajoutEspaceAdministrateur	3 days ago		8 1	

Une fois les développements terminés et testés localement, nous avons effectué des merge requests vers la branche dev afin d'intégrer les nouvelles fonctionnalités de manière sécurisée et contrôlée. Après validation et vérification du bon fonctionnement global, les modifications ont été fusionnées sur la branche main pour garantir une version stable et déployable de l'application.

Cette organisation nous a permis de collaborer efficacement, d'assurer un suivi clair des tâches réalisées par chacun et de maintenir une base de code cohérente et sécurisée tout au long du projet.

6. Sécurité du système

6.1. Protection de la connexion contre les attaques force brute

Pour protéger le site face aux potentiels attaques par brute force, nous avons installé le module rate-limiter, permettant de gérer ceci.

Ainsi nous avons juste eu à rajouter, dans services.yaml le code ci dessous

```
login_throttling:
    max_attempts: 3
    interval: '15 minutes'
```

6.2. Politique de mot de passes et exigences de sécurité

Comme demandé dans les consignes, nous avons prit les recommandations de sécurités de l'ANSSI pour le mot de passe en rajoutant à ceci la non compromission et la non réutilisation.

Pour ceci nous avons dû activer dans validator.yaml (avec l'extension validator de Symfony)

et voilà, le mot de passe ne peut pas être compromis en lien avec le site <https://haveibeenpwned.com> relevant les mots de passes compromis lors de failles de sécurités.

Ensuite nous avons fais le forms du register et pour changer le mot de passe avec ceci:

```
->add('plainPassword', PasswordType::class, [
    // instead of being set onto the object directly,
    // this is read and encoded in the controller
    'mapped' => false,
    'attr' => ['autocomplete' => 'new-password'],
    'constraints' => [
        new NotBlank([
            'message' => 'Veuillez saisir un mot de passe.',
        ]),
        new Length([
            'min' => 12,
            'minMessage' => 'Votre mot de passe doit contenir au moins {{ limit }} caracteres.',
            // max length allowed by Symfony for security reasons
            'max' => 4096,
        ]),
        new Regex([
            'pattern' => '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[\\W_]).+$/ ',
            'message' => 'Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.',
        ]),
        new NotCompromisedPassword([
            'message' => 'Ce mot de passe a ete compromis dans une fuite de donnees. Merci d\\'en choisir un autre.',
        ]),
    ],
```

Nous pouvons voir que les vérifications sont: la taille du mot de passe, les regex (minimum une minuscule, une majuscule, un chiffre et un caractère spécial) et si il n'est pas compromis pour l'inscription (il n'a donc pas d'ancien mot de passe sur le site)

```
private function assertPasswordIsNew(User $user, string $plainPassword): void
{
    $hasher = $this->getHasherForUser($user);

    foreach ($user->getPasswordHistory() as $password) {
        if ($hasher->verify($password->getHash(), $plainPassword)) {
            throw new PasswordReuseException('Password has been used before.');
        }
    }
}
```

et ci dessus, demandé par le controlleur, la fonction vérifiant si le password n'a pas déjà été utilisé par l'utilisateur.

6.3. Gestion de la réinitialisation de mot de passe

Pour répondre à cette problématique, nous avons créé un page “profile” qui récupérera les données de la personne connecté, ensuite nous avons inséré un formulaire de modification dans cette même page pour y changer le mot de passe.

```
if ($passwordForm->isSubmitted() && $passwordForm->isValid()) {
    $currentPassword = (string) $passwordForm->get('currentPassword')->getData();
    $newPassword = (string) $passwordForm->get('newPassword')->getData();

    if ($currentPassword === $newPassword) {
        $passwordForm->get('newPassword')->addError(new FormError('Votre nouveau mot de passe doit etre differe
    } else {
        try {
            $passwordManager->changePassword($user, $currentPassword, $newPassword);
            $this->addFlash('success', 'Votre mot de passe a ete mis a jour.');
```

Mon profil

Informations personnelles	Changer de mot de passe
Prenom Tetz	Le nouveau mot de passe doit contenir au minimum 12 caracteres et ne pas avoir ete utilise auparavant.
Nom tetsts	Mot de passe actuel
Adresse zadad	Nouveau mot de passe
Ville azeazea	Confirmez le mot de passe
Code postal 17539	Mettre a jour le mot de passe
Mettre a jour le profil	

mon profil

Votre mot de passe a ete mis a jour.



Informations personnelles

Prenom

Nom

Adresse

Ville

Code postal

Mettre a jour le profil

Changer de mot de passe

Le nouveau mot de passe doit contenir au minimum 12 caracteres et ne pas avoir ete utilise auparavant.

Mot de passe actuel

Votre mot de passe actuel est incorrect.

Nouveau mot de passe

Confirmez le mot de passe

Mettre a jour le mot de passe

Puis ensuite, nous avons la fonction qui permet de vérifier si le nouveau mot de passe à déjà été utilisé auparavant, un message d'erreur s'affiche alors si celui-ci a déjà été utilisé.

```
return $this->redirectToRoute('app_profile');  
} catch (InvalidCurrentPasswordException) {  
    $passwordForm->get('currentPassword')->addError(new FormError('Votre mot de passe actuel est incorrect'));  
} catch (PasswordReuseException) {  
    $passwordForm->get('newPassword')->addError(new FormError('Vous ne pouvez pas reutiliser un ancien mot de passe'));  
}
```


Vous ne pouvez pas réutiliser un ancien mot de passe.



Informations personnelles

Prenom

qsd

Nom

dsq

Adresse

1 qsd

Ville

sqd

Code postal

54878

Mettre à jour le profil

Changer de mot de passe

Le nouveau mot de passe doit contenir au minimum 12 caractères et ne pas avoir été utilisé auparavant.

Mot de passe actuel

Nouveau mot de passe

Confirmez le mot de passe

Mettre à jour le mot de passe

7. Test et validation

7.1. Tests Fonctionnels

Pour faire les tests fonctionnels, nous avons parcouru nos user stories afin d'essayer de voir les erreurs et d'avoir un parcours utilisateur viable.

exemple : Passage du Prénom de l'utilisateur qsd en Jean-pierre

dsq qsd

Adresse email

thibault.royer59@hotmail.fr

Rôles

☒ Utilisateur ☒ Administrateur ☐ Modérateur

☒ Email vérifié

Prénom

qsd

Nom

dsq

Adresse postale

1 qsd

Code postal

54878

Ville

sqd

Date de création

19/11/2025

☐ Utilisateur suspendu ?

Modifier

dsq Jean-Pierre

Ou même la connexion

Connexion

Email

thibault.royer59@hotmail.fr

Mot de passe

••••••••••

[Mot de passe oublié ?](#)

☐ Se souvenir de moi

Se connecter

Pas encore de compte ? [Inscrivez-vous.](#)

- en haut à droite dans la navBar

[Accueil](#) [Listes des utilisateurs](#) Jean-Pierre ▾

la modification du mot de passe:

Changer de mot de passe

Le nouveau mot de passe doit contenir au minimum 12 caractères et ne pas avoir été utilisé auparavant.

Mot de passe actuel

••••••••••

Nouveau mot de passe

••••••••••

Confirmez le mot de passe

••••••••••

Mettre à jour le mot de passe

Votre mot de passe a ete mis a jour.



Informations personnelles

Prenom

Jean-Pierre

Nom

dsq

Adresse

1 qsd

Ville

sqd

Code postal

54878

Mettre a jour le profil

Changer de mot de passe

Le nouveau mot de passe doit contenir au minimum 12 caracteres et ne pas avoir ete utilise auparavant.

Mot de passe actuel

Nouveau mot de passe

Confirmez le mot de passe

Mettre a jour le mot de passe

les logs de connexions :

logs



utilisateur	id_utilisateur	date	navigateur	ip
Jean-Pierre	1	16:11:08 19/11/2025	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 OPR/123.0.0.0	10.0.217.185
Jean-Pierre	1	18:11:08 19/11/2025	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 OPR/123.0.0.0	10.0.217.185
Jean-Pierre	1	51:11:11 20/11/2025	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 OPR/123.0.0.0	fd42:4aef:1bb2:d73f:1266:6aff:fe61:6259
Jean-Pierre	1	19:11:11 20/11/2025	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 OPR/123.0.0.0	fd42:4aef:1bb2:d73f:1266:6aff:fe61:6259
Jean-Pierre	1	44:11:12 20/11/2025	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 OPR/123.0.0.0	fd42:4aef:1bb2:d73f:1266:6aff:fe61:6259

Nous venons de remarquer un problème avec les dates, où l'heure exède la journée

7.2. Tests de sécurité

Changement de mot de passe / entrée d'un mot de passe sur le site invalide

Podfqsdqsdsq5566 -> manque un caractère spécial

Nouveau mot de passe



Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.

poqsdqsdqsd366. -> manque une majuscule

Nouveau mot de passe

Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.

QSDSQDHQSHD5665. -> manque une minuscule

Nouveau mot de passe

Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.

qsdqSdsqd..qsd -> manque un chiffre

Nouveau mot de passe

Votre mot de passe doit contenir au moins une minuscule, une majuscule, un chiffre et un caractere special.

d44???qQ -> trop court

Nouveau mot de passe

Votre mot de passe doit contenir au moins 12 caracteres.

MotDePasse1. -> compromis dans une fuite de donnée

Nouveau mot de passe

Ce mot de passe a ete compromis dans une fuite de donnees. Merci d'en choisir un autre.

Et maintenant, la protection contre le brute force :

Après trois entrées fausse sur la connexion :

Connexion

Too many failed login attempts, please try again
in 15 minutes.

Email

test@test.test

Mot de passe

.....

[Mot de passe oublié ?](#)

☐ Se souvenir de moi

Se connecter

Pas encore de compte ? [Inscrivez-vous.](#)

Conclusion

Ce projet avait pour objectif de concevoir une application Symfony sécurisée permettant de gérer les inscriptions, connexions et habilitations des utilisateurs tout en respectant les bonnes pratiques de cybersécurité. L'ensemble des exigences clés a été respecté : validation des informations saisies grâce aux contraintes Symfony, gestion sécurisée des mots de passe avec vérification de la complexité et interdiction de réutilisation des anciens mots de passe, protection contre les attaques par force brute, et suivi complet de l'historique des connexions.

Le système inclut également un module administrateur permettant de gérer les comptes utilisateurs, les rôles et les habilitations (ajout, modification, suppression et suspension), tout en garantissant la sécurité et la fiabilité des opérations. L'application est hébergée sur GitHub et conçue de manière modulaire pour pouvoir servir de base à de futurs projets.

Ce projet nous a permis de renforcer nos compétences en développement Symfony, en sécurité des applications et en gestion des accès. Il nous a également apporté une meilleure compréhension de la conception d'un système sécurisé, fiable et maintenable.

En conclusion, la solution développée répond pleinement aux objectifs fixés : elle assure une authentification sécurisée, une gestion rigoureuse des utilisateurs et des rôles, tout en respectant les standards de cybersécurité.

Ressources documentaires et références utilisés

Documentation utilisées

- [Symfony Documentation – Documentation officielle complète sur le framework Symfony, incluant la gestion des formulaires, la sécurité, les rôles et habilitations.](#)
- [Symfony Security Component – Guide détaillé sur la mise en place de l'authentification, le contrôle d'accès et la protection contre les attaques courantes.](#)
- [Symfony Validators – Documentation sur la validation des données et l'utilisation des contraintes pour sécuriser les saisies utilisateurs.](#)
- [Symfony Password Hasher – Ressource sur la gestion sécurisée des mots de passe \(hachage, complexité, non-réutilisation\).](#)

Ressources pédagogiques et cours utilisés

- Cours EPSI / Module Symfony
- Cours EPSI / Mobile avec Flutter SLAM SN2
- Cours EPSI / Préparation Epreuve E6 : Cybersécu
- [NotCompromisedPassword Validator Symfony – Référence pour la validation de mots de passe afin de prévenir l'utilisation de mots de passe compromis.](#)

Annexes

Annexe 1

Le repository github avec nos différentes branches de travail des fonctionnalités

Branches

New branch

OverviewYoursActiveStaleAll

Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	1 hour ago	3 / 3			Default

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
dev	6 hours ago	2 / 3	2	0	

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
features/loginLogs	1 hour ago	4 / 5	1	0	#5
dev	6 hours ago	2 / 3	2	0	#6
features/ajoutUpdatePassword	6 hours ago		8	1	#4
features/ajoutEspaceAdministrateur	2 days ago		8	1	#2

Annexe 2

Notre méthode SCRUM

☐	Tickets	Personne assignée	Rapporteur	
☐	☒ SCRUM-6 L'application doit être hébergée sur GitHub	FG Florian Guilbert	TR Thibault Roy	
☐	☒ SCRUM-5 L'administrateur doit pouvoir gérer les com...	TR Thibault Royer	TR Thibault Roy	
☐	☒ SCRUM-4 Le système de connexion doit être protégé ...	TR Thibault Royer	TR Thibault Roy	
☐	☒ SCRUM-3 utilisateur doit pouvoir modifier son mot de ...	FG Florian Guilbert	TR Thibault Roy	
☐	☒ SCRUM-2 mot de passe doit respecter l'ensemble des...	Kaelian BAUDELET	TR Thibault Roy	
☐	☒ SCRUM-1 s'inscrire ou se connecter	Kaelian BAUDELET	TR Thibault Roy	
+ Créer 6 sur 6 ↺				

Annexe 3

User stories

 Ajouter epic / ☒ SCRUM-1

s'inscrire ou se connecter

+

✓ Description

Page d'inscription et de connexion en utilisant Validation Constraints
Reference de Symfony pour valider les saisies avant leur insertion
dans la bdd

 Ajouter epic / ☒ SCRUM-2

mot de passe doit respecter l'ensemble des critères de sécurité préconisés

+

✓ Description

L'utilisateur rentre un mot de passe en phase avec les
recommandations de CNIL.

 Ajouter epic / ☒ SCRUM-3

utilisateur doit pouvoir modifier son mot de passe SANS utiliser un ancien mot de passe

+

✓ Description

L'utilisateur souhaite changer son mot de passe cependant sans
utiliser d'anciens mots de passes.

 Ajouter epic / ☒ SCRUM-4

Le système de connexion doit être protégé contre les attaques par force brute.



✓ Description

Le système doit pouvoir résister à ces attaques par forces brutes pour en améliorer la sécurité

 Ajouter epic / ☒ SCRUM-5

L'administrateur doit pouvoir gérer les comptes, rôles et habilitations



✓ Description

L'administrateur peut aller sur la page d'utilisateur et changer les permissions des utilisateurs.

 Ajouter epic / ☒ SCRUM-6

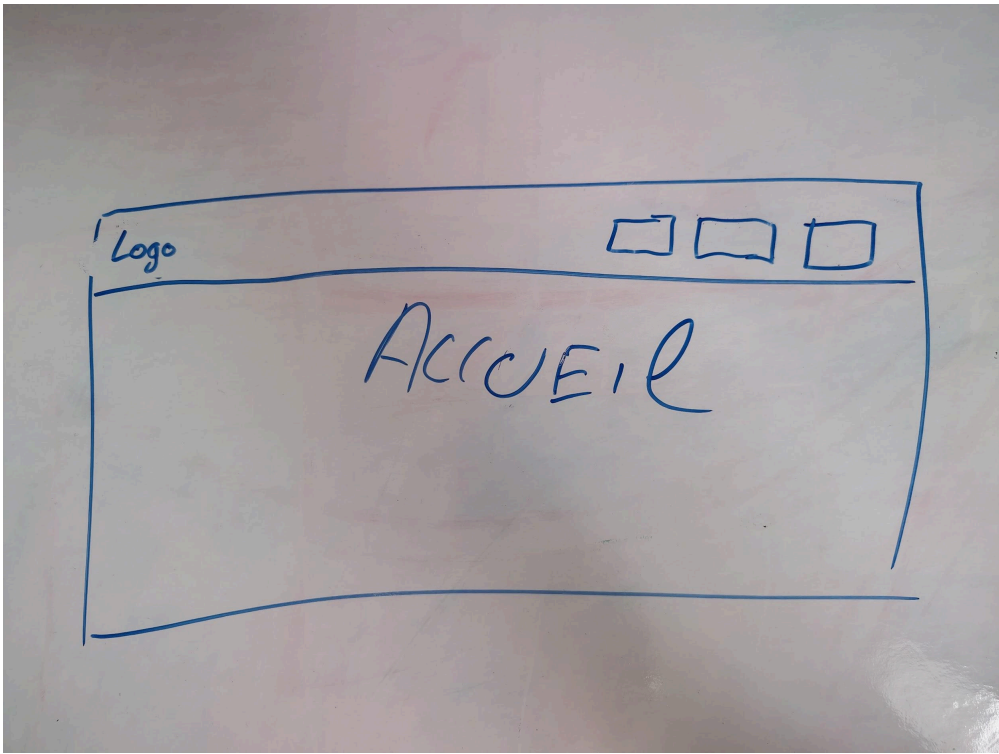
L'application doit être hébergée sur GitHub



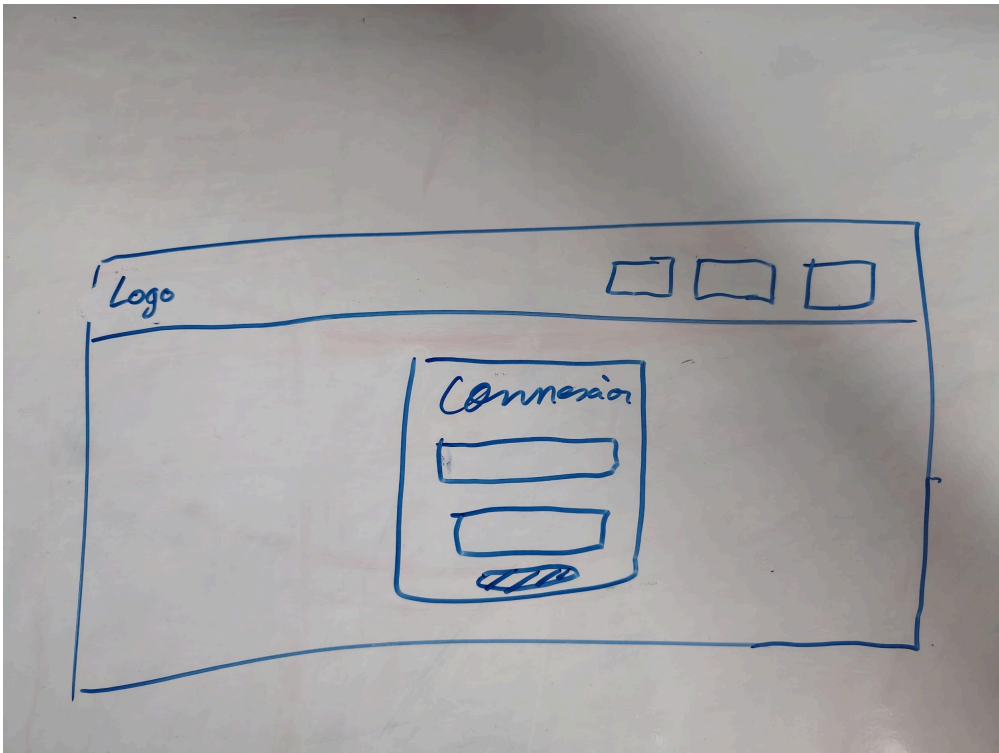
✓ Description

Hébergement de l'application sur github

Annexe 4



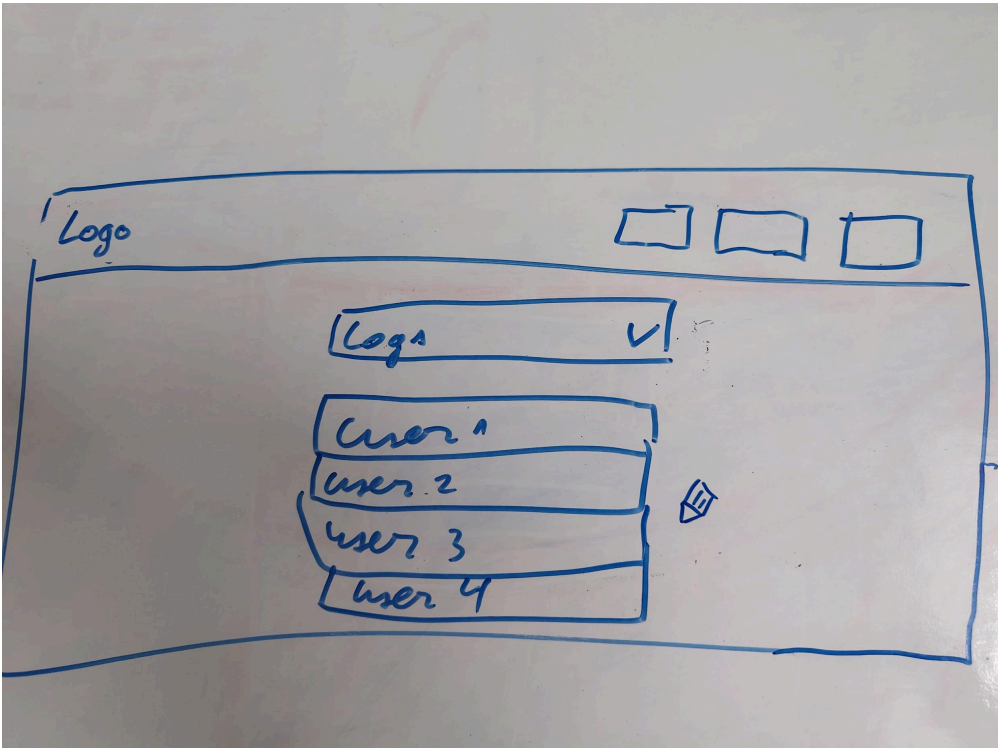
Annexe 5



Annexe 6



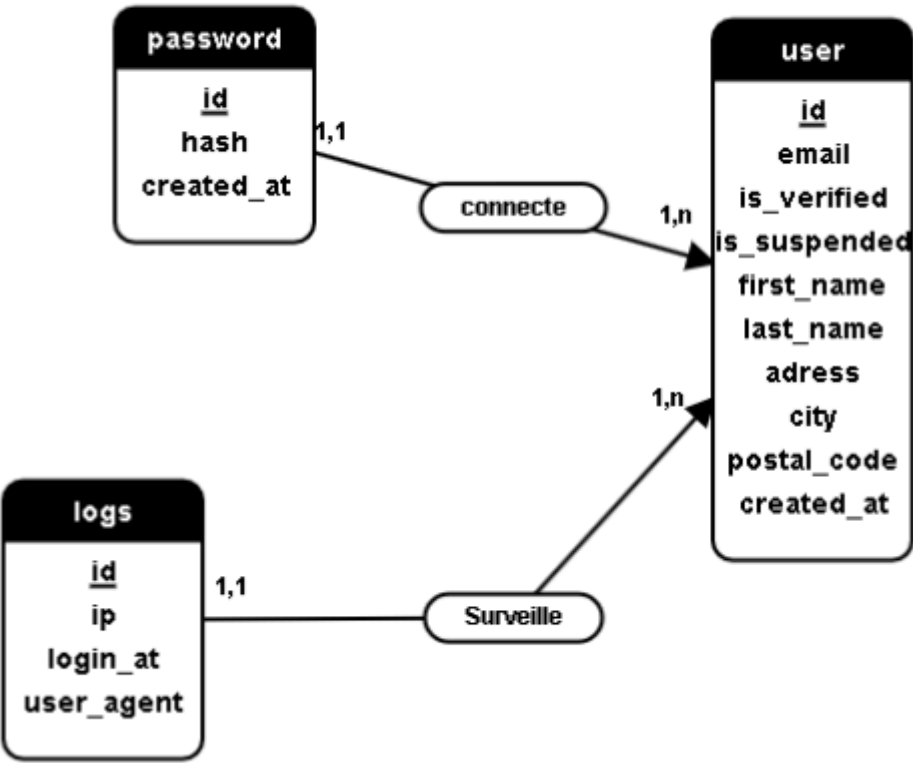
Annexe 7



Annexe 8

TABLES	NOM DE DONNÉES	DESCRIPTION	NATURE	TYPE DE DONNÉES	FORMAT	TAILLE	CONTRAINTES
user	id	identifiant de l'utilisateur	Elementaire	Numérique		-	ID
user	lastname	Nom de l'utilisateur	Elementaire	Varchar		50	Not-Null
user	firstname	Prénom de l'utilisateur	Elementaire	Varchar		50	Not-Null
user	adress	Adresse de l'utilisateur	Elementaire	Varchar		255	Not-Null
user	city	Ville de l'utilisateur	Elementaire	Varchar		50	Not-Null
user	postalCode	Code postal de l'utilisateur	Elementaire	Numérique		5	Not-Null
user	role	Rôles de l'utilisateur	Elementaire	longtext		-	Not-Null
user	creationDate	Date de création du compte de l'utilisateur	Elementaire	Date	AAAA-MM-DD HH:MM:SS	-	Not-Null
user	email	email de l'utilisateur	Elementaire	Varchar		50	Not-Null
password	id	identifiant du password	Elementaire	Numérique		-	ID
password	hash	hash du password (le password n'est pas stocké en clair)	Elementaire	Varchar		255	Not-Null
password	created_at	date à laquelle le password a été enregistré	Elementaire	Date	AAAA-MM-DD HH:MM:SS	-	Not-Null
logs	id	identifiant du log	Elementaire	Numérique		-	ID
logs	ip	ip de la personne s'étant connectée	Elementaire	Varchar		255	Not-Null
logs	login_at	date à laquelle l'utilisateur s'est connecté	Elementaire	Date	AAAA-MM-DD HH:MM:SS	-	Not-Null
logs	user_agent	navigateur sur lequel l'utilisateur s'est connecté	Elementaire	Varchar		255	Not-Null

Annexe 9



Annexe 10

dsq qsd

Adresse email

thibault.royer59@hotmail.fr

Rôles

☒ Utilisateur

☒ Administrateur

☐ Modérateur

☒ Email vérifié

Prénom

qsd

Nom

dsq

Adresse postale

1 qsd

Code postal

54878

Ville

sqd

Date de création

19/11/2025

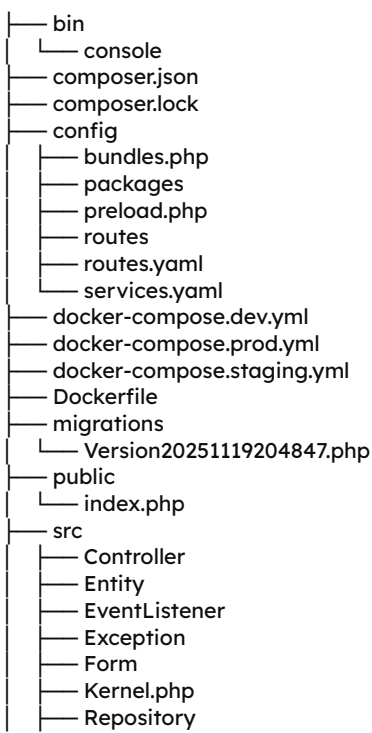
☐ Utilisateur suspendu ?

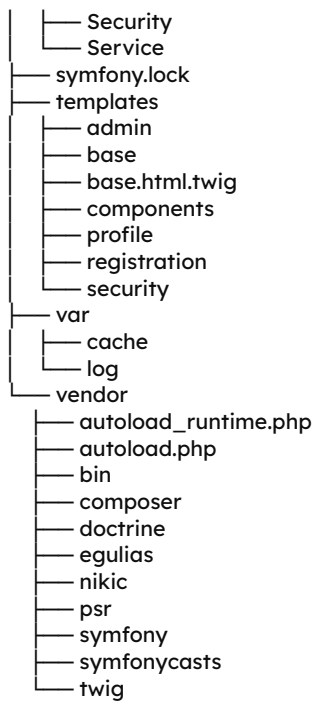
Modifier

Annexe 11

Structure du projet

projet-cybersecurite-ap2





Le présent document est exclusivement destiné à un usage pédagogique dans le cadre scolaire. Il s'inscrit dans un contexte strictement éducatif et non public.

Toute utilisation, reproduction, diffusion ou exploitation, totale ou partielle, à des fins autres que l'apprentissage notamment à des fins commerciales, professionnelles ou promotionnelles est formellement interdite et pourra faire l'objet de poursuites conformément aux dispositions en vigueur relatives à la propriété intellectuelle. (voir notamment l'article Article L122-5 du Code de la propriété intellectuelle)